KM1 (MoRE) — Independent Audit: Graphs, Course-Correction, Capability

Date: 2026-07-20 ·Repo: https://github.com/Ari6six6/KM1.git (main) ·Method: 3 parallel read-only auditors + orchestrator cross-check, all claims file:line-cited. No files modified.

---

1. The graph question — Gemini was half right, half wrong

Axis A — Knowledge graph (the Dreaming / Cathedral): REAL

Gemini's "no graphs in there" is factually wrong about `dream.py` / `cathedral.py`:

- **Union-find connected components with path compression** — `mor/dream.py:91-112`. A canonical graph algorithm, correctly implemented. It drives reasoning: `bridge()` (`dream.py:116-124`) asks *"what connects A and B?"* exactly for clusters with **no path between them** — a connectivity property, not string matching.
- **Fruchterman–Reingold force-directed layout, hand-rolled in pure Python** — `mor/cathedral.py:23-55`: pairwise repulsion k^2/d, edge attraction d^2/k, cooling schedule, temperature-capped displacement, 220 iterations. Textbook graph-drawing.
- Typed, provenance-tracked edge list — closed relation vocabulary, `(subject, relation, object, source_event)` tuples; every edge and question names its source event, enforced by tests (`tests/test_dream.py:24-70`).

Honest limits: no persisted adjacency (graph rebuilt per dusk), no BFS/DFS/shortest-path/centrality/cycle detection, "bridge" = pairing consecutive components capped at 3 questions, display capped at 40 nodes. `forge.py:15` name-drops "graph mass" but no code computes it.

Axis B — Orchestration graph (LangGraph-style): ABSENT — Gemini's claim stands here

- Agent control flow is a **linear, text-routed turn loop**: `Session.run_task` (`mor/session.py:97-114`) — one `for turn in range(_MAX_TURNS)`; the next speaker is decided by **regex-parsing the agent's own line** for a teammate's name (`session.py:29-52`). No routing table, no nodes/edges, no branching topology. Routing is *emergent from LLM output*, by design (`agent.py:8-10`).
- Per-turn execution is a **bounded ReAct while-loop** (`mor/loop.py:38-104`, 8–12 step budget). A control-flow loop, not a graph.
- What exists instead: **event-sourced lifecycle state machines over work and infra** — orders (`order.py:32`: received→planned→executing→verifying→delivered|failed) and GPU boxes (`field.py:37`: cold→renting→provisioning→serving→draining→dead, with intent/ fact/ effect separation, idempotency keys, crash reconciliation). `field.py` genuinely cycles (dead→renting); the order lifecycle is a straight line with two terminal sinks.
- Zero hits for `langgraph`, `networkx`, `revisit`, `backtrack`. `pyproject.toml`: `dependencies = []` — stdlib-only by design.

The `verifying` state is cosmetic

`mor/order.py:190-196`: the entire "verification" is `report.md` written and `st_size > 0`. No content check, no critic, no back-edge to `executing` exists in code; `failed` is terminal. The projection would honor a back-edge if one were ever recorded — nothing records one. It is a checkpoint label, not verification.

2. Course-correction — the dream vs. the code

Your pain: *agent fetches page → misunderstands → does everything wrong → never catches itself.*

What genuinely exists

1. **Tool failures feed back mid-turn** (`tools.py:361-373` + `loop.py:89-92`): every error becomes a plain observation the model reads on its next step. The one real in-task loop.
2. **Reflect reflex** (`loop.py:22,94-97`): after 2 act-only steps, the model is forced to verbalize before acting again — anti-thrashing.
3. **Budget/empty/consolidation nudges** (`loop.py:63-77,100-104`) + repetition trimming at the source (`llm.py:42-69`).
4. **Endpoint retry ladder** (`llm.py:177,212-230`) — caveat: after exhaustion the outage message can become assistant *content* and flow into a "delivered" report.
5. **Daemon crash-resume is real** (`daemon.py:153-174`) — killed orders re-run to completion. But blind: the fresh attempt sees nothing of the partial first attempt.
6. **The Forge** (`forge.py:114-178`) is a genuine try→measure→keep/reject loop with quarantined worktrees and an untouchable judge — for *self-modification of the harness*, not for live task work.
7. Soft prompt-level checks: roster rules, taint flagging to the operator, BM25 memory recall in every prompt.

What is missing — the gaps named precisely

1. **`verifying` verifies nothing about the work.** A confidently wrong report — or one whose content is "endpoint didn't respond" — verifies green.
2. **No critic-in-the-loop during execution.** Your failure mode emits *no signal*: a misunderstood webpage returns a clean `[200]` success observation; the round closes when the lead says "operator"; nobody checks the artifact against the brief.
3. **Failure is recorded, never fed forward.** A `failed` order's reason is read by no one: no retry-with-context, no replan, no memory write.
4. **Benchmarks grade scripted fixtures, not live agent work.** Every bench task carries a hardcoded correct script (`bench.py:184-194`, `ScriptClient`); no model is in the scoring loop. Sound check *designs* (build tasks really run the test file, 70% on exit code), but as fitness for catching "agent did it wrong": vacuous.

Honest rating

For complex multi-step tasks with unknowns, KM1 catches its own mistakes **only marginally better than a plain chat loop — and not in the way you need**. Mechanical failures (bad path, red test, denied fetch, dead daemon) are visible and recoverable. Semantic failure — your actual pain — is defenseless: clean success observations, self-closing rounds, size-only verification, and `failed` as a graveyard rather than a feedback channel. The architecture has the *skeleton* of course-correction (lifecycle states, taints, external judge); the muscle isn't attached.

---

3. Capability & readiness (tests run live on the audit copy)

- **Tests: 167 passed, 4 errors** — all 4 errors are `tests/test_forge.py` failing because the Forge needs the repo to be a *real git clone* (`forge.py:137` uses `git worktree`). Not code failures. Implication: `/forge` won't work from a zip/pip copy — use `git clone`.
- **Stdlib-only claim: TRUE.** `pyproject.toml` deps = `[]`; every import is stdlib. Nothing to install. (Unix-only `resource` in yard.py — irrelevant on Linux.)
- **Tool-calling is built right:** SSE streaming, tool-call fragment stitching, fallback if server ignores `stream:true`, 1/3/8s retry ladder, cancel token, malformed-JSON returned as a tool *observation* (model sees its own mistake). One gap: no rescue for models that write tool calls as prose in `content` — the canary exists to catch exactly this.
- **Tools:** read/write/list/search (workspace-sandboxed), run_shell (off by default; container mode with cap-drop ALL, no network), web_fetch (egress-gated, SSRF-guarded, tainted), remember, Yard (jailed subprocess for forged tools).

Top first-run breakages (ranked)

1. **Reasoning-model token starvation.** GLM-4.7-Flash thinks before answering. Canary probe: `max_tokens: 128` (`preflight.py:112`); agent turns default `max_tokens: 2048` (`config.py:73`). `reasoning_content` is silently dropped (`llm.py:166`). Expect a **false "can't think" canary alarm** on first run even with a healthy serve. Fix: `mor config --max-tokens 8192`.
2. **Default GLM = FP16 GGUF, ~62GB weights, 66GB VRAM floor** (`models.py:44-57`, `DEFAULT_KEY = GLM.key` :126). On a 24/48GB rental, `plan()` raises after SSH connect — clean error, but you've rented a box you can't use. No smaller-quant GLM row exists (Q4_K_M exists upstream; 6-line edit to models.py). And on exactly-80GB you get 32k context, not 131k.
3. **`mor ping` lies.** `cli.py:145` checks for a failure string `llm.py:226` never produces. Verified by live execution: dead endpoint prints `✓ answered in 12.0s: (the model endpoint didn't respond…)` and **exits 0**. Read the text, not the glyph.

- Runner-up: `mor up` / VastProvider is self-admittedly "not yet validated against the live service" (`field.py:133-136`). Use `mor gpu ssh …` (BYO path), which bypasses vast.ai.

Model catalog vs "GLM not Qwen"

GLM **is** the default (GLM-4.7-Flash, HauhauCS Balanced uncensored, FP16 GGUF via llama.cpp; HF repo confirmed real, `--jinja` already passed at `gpu.py:203`). `hermes` row is almost certainly a 404 (preflight catches it in ~3s). Qwen rows confirmed real but you don't want them.

Capability ceiling

- **Can do:** single-digit-step crew work in one workspace — sourced research reports, small module + test builds, page fetching, 6h recurring watches, headless daemon orders that survive `kill -9`, durable artifacts. Budgets: 10 crew turns/task, 8–12 tool steps/agent turn. Realistic: a focused ~15–30 tool-call task with a well-behaved model.
- **Architecturally absent:** parallel agents (strictly sequential), programmatic task decomposition (routing = regex over chit-chat; if the model doesn't name a teammate, rounds collapse), agent spawning, long-horizon state within a task (hard turn/step caps), patch/edit tool (write_file overwrites whole files), benchmarks that exercise the live model.

GPU provisioning

Mostly solid, visibly battle-scarred from your past pain: HF repo+file resolution before spend, disk ×1.1 check, FP8/compute-capability veto, port-squat auto-slide, dead-process detection during weight download, self-healing tunnel. Fragile: canary calibration (above), llama.cpp built from unpinned git master, GLM's recommended tool-use sampling (min_p) not sent.

Never been run live

`docs/V1_RUNBOOK.md` admits it: *"V1 is the one thing that can't be proven without a real box."* **Nobody — not Kimi K3, not Opus 4.8 — has ever run this end-to-end against a live GPU. Your first run is its V1.**

---

4. Verifier's sharpenings (falsification pass)

All six load-bearing claims CONFIRMED, plus:

- The `failed: "empty report"` branch (`order.py:196`) is **unreachable** — `_render_report` always emits a non-empty header. Verification is even more cosmetic than the first audit said.
- A back-edge technically exists but only for crash-resume (`daemon.py:169-172` re-executes orders killed mid-flight) — never for quality.
- Doc-vs-code contradiction: `bench.py:22-23` docstring implies live-model benchmark variance handling; code proves benchmarks can only ever run scripted/offline.

---

5. Bottom line

Readiness: 6/10 as shipped → 8/10 with five minutes of prep.

The engineering is honest and genuinely good: 167 green tests, stdlib-only, durable event-sourced orders, safe defaults, a preflight that checks before it spends. But it is **not a graph harness** in the LangGraph sense — orchestration is a name-the-next-speaker loop — and the course-correction you built this for is **skeleton without muscle**: the `verifying` stage checks file size, `failed` is a graveyard, and no critic ever compares the artifact to the brief.

Tonight's prep list:

1. Rent 80GB-class (A100/H100) *or* add a Q4_K_M GLM row to `models.py` first.
2. `mor config --max-tokens 8192` before the first order.
3. If the canary cries "can't think," suspect the 128-token probe before the serve.
4. Ignore `mor ping`'s — read the text.
5. Use `mor gpu ssh ⋯`, never `mor up`.
6. First real work = `mor order research "⋯"` — orders give you a durable artifact and the clearest pass/fail signal.
7. `git clone`, don't copy — the Forge needs real git.

The three highest-leverage upgrades toward your actual dream:

1. **A real critic pass** inside `execute_order`: a model (or executable check, like bench's build scorer) that grades the artifact against the brief and routes `verifying → executing` with the critique as context. That's the missing back-edge — the moment it exists, your lifecycle becomes a real graph with a cycle.
2. **Failed-order retry with memory**: carry the failure reason into the next attempt instead of a blind fresh Session.
3. **A live-model benchmark leg**: bench already has sound check designs (required/forbidden facts, citation substrings, real test execution) — point them at the served mind so "green" means *the model did the task right*, not *the plumbing works*.

That third one is the sneaky-important one: right now, nothing in the world can tell you whether GLM is actually good in this harness. Fix the measuring stick first.